VexenLabs Server Recovery

Hardened rebuild + DoctorFinder MVP deployment

Operator	Aslam Khan (m8826825669)
Server	DigitalOcean droplet — vexenlabs-prod-01 (68.183.108.180)
OS	Ubuntu 24.04 LTS
Date	May 14–15, 2026
Outcome	DoctorFinder live at https://doctor.vexenlabs.com

This document records the full sequence of actions taken to recover from a compromised production droplet and stand up a hardened replacement with a working DoctorFinder demo. It is intended as a future-reference runbook — if any of these issues recur, the fixes are documented here in order.

1. The Compromise — What Happened

On May 14, 2026, the original DigitalOcean droplet (157.245.248.210) was found running a cryptominer. The compromise was discovered when the droplet's CPU usage stayed pegged after all application traffic had stopped.

1.1 Indicators of compromise

After investigation, the following malicious artefacts were found running as the login user **aslam**:

Component	Detail
Kinsing + kdevtmpfsi cryptominer	Running at 28.9% CPU, ~151 minutes total CPU time consumed
Reverse shells	Outbound connections to 107.175.89.136:9009 (ColoCrossing C2)
Cron persistence	Pulled unk.sh from 80.64.16.241 every 60 seconds
Shell tampering	aslam's .bashrc had a hook causing non-interactive shells to exit
Likely root persistence	Root SSH was enabled; root account may have been compromised

1.2 Root cause

The compromise vector was most likely a Next.js 15.1.3 RCE (CVE-2025-29927) combined with two operational mistakes that amplified blast radius:

• The Node.js service ran as the login user **aslam**, giving the attacker shell-level filesystem access on a successful exploit.

• The droplet had **no outbound firewall**, so the reverse shell and crypto-mining traffic flowed freely.

1.3 Decision: destroy, not clean

Cleaning a compromised host is not safe — kernel rootkits, hidden users, and persistence backdoors cannot be reliably enumerated. The decision was made to:

1. Take a DigitalOcean snapshot named **compromised-evidence-2026-05-14** for forensic preservation.
2. Pull evidence files via SSH-as-root (since aslam's shell was tampered).
3. Destroy the droplet entirely.
4. Build a fresh, hardened replacement.
5. Rotate every credential the old droplet had access to.

2. Building the Hardened Replacement

The new droplet (68.183.108.180, 1GB Ubuntu 24.04) was built with security hardening applied *before* any application services were installed.

2.1 SSH hardening

Created a non-login **deploy** user with sudo, copied SSH key for passwordless auth, then locked down sshd:

```
# /etc/ssh/sshd_config.d/99-hardening.conf
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers deploy
```

2.2 Firewall — deny by default OUTBOUND

UFW was configured to deny outbound traffic by default — a key improvement that would have blocked the malware's C2 callback:

```
sudo ufw default deny incoming
sudo ufw default deny outgoing
# Inbound
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
# Outbound (only what's necessary)
sudo ufw allow out 53      # DNS
sudo ufw allow out 80/tcp  # apt, certbot
sudo ufw allow out 443/tcp # apt, certbot, GitHub
sudo ufw allow out 123/udp # NTP
sudo ufw allow out 587,465/tcp  # email
sudo ufw allow out on lo
sudo ufw enable
```

Verification: confirmed outbound port 9009 to the old C2 IP was blocked.

2.3 Other hardening

• **fail2ban** with sshd jail (3 max retries, 24h ban)

• **unattended-upgrades** enabled for automatic security patches

• Kernel hardening via /etc/sysctl.d/99-hardening.conf: SYN cookies, dmesg_restrict, unprivileged_bpf_disabled, fs.protected_symlinks, etc.

• Snapshot **hardened-baseline-2026-05-14** taken before any application install.

3. Application Infrastructure

3.1 Service users — isolation pattern

Each app got its own non-login system user, with a directory pattern that lets the **deploy** user write and the service user only read:

```
sudo useradd -r -s /usr/sbin/nologin -d /nonexistent doctor-api
sudo useradd -r -s /usr/sbin/nologin -d /nonexistent doctor-web

sudo mkdir -p /var/www/doctor/{backend,frontend,logs,media,staticfiles}
sudo chown -R deploy:doctor-api /var/www/doctor/backend
sudo chown -R deploy:doctor-web /var/www/doctor/frontend
sudo chmod -R 750 /var/www/doctor
sudo find /var/www/doctor -type d -exec chmod g+s {} \;
```

> The setgid bit on directories ensures new files inherit the group, so the service user can always read newly created files without manual chmod.

3.2 Stack components

Component	Version	Purpose
PostgreSQL	16.13	Application database (localhost only)
Redis	7.x	Cache + session storage (password-protected)
Python	3.12.3 + uv	FastAPI runtime
Node.js	22.22.2 + pnpm 11	Frontend tooling
nginx	1.24	Reverse proxy + static file server
Certbot	latest	Let's Encrypt SSL certs (auto-renew)

3.3 Database isolation

Three databases were created with cross-database access revoked from PUBLIC — each app user can only see its own database:

```
CREATE DATABASE vexen_marketplace OWNER vexen_api;
CREATE DATABASE hms_db OWNER hms_api;
CREATE DATABASE doctorfinder_db OWNER doctor_api;

REVOKE CONNECT ON DATABASE vexen_marketplace FROM PUBLIC;
REVOKE CONNECT ON DATABASE hms_db FROM PUBLIC;
REVOKE CONNECT ON DATABASE doctorfinder_db FROM PUBLIC;
```

3.4 DNS + SSL

DNS A records pointed at Hostinger:

```
@         A   68.183.108.180   (vexenlabs.com)
www       A   68.183.108.180
api       A   68.183.108.180
hospital  A   68.183.108.180
hapi      A   68.183.108.180
doctor    A   68.183.108.180
drapi     A   68.183.108.180
```

SSL certs issued via Certbot with auto-renewal:

```
sudo certbot --nginx -d vexenlabs.com -d www.vexenlabs.com -d api.vexenlabs.com
sudo certbot --nginx -d doctor.vexenlabs.com
```

4. Bugs Encountered (and How They Were Fixed)

During DoctorFinder deployment, 18+ bugs surfaced. They are catalogued here so anyone redoing this deploy can skip the debugging time.

4.1 Backend bugs (dr-finder-python)

Missing alembic.ini

Repo shipped without alembic.ini. alembic upgrade head failed with 'No config file found'. Fix: created alembic.ini with sqlalchemy.url empty (loaded from app.config at runtime).

Missing migrations/env.py

Same root cause as above. Created env.py reading DATABASE_URL_SYNC from settings, using psycopg2 (sync) for migrations even though the app uses asyncpg at runtime.

psycopg2 not in requirements.txt

Migration tried to import psycopg2; only asyncpg was listed. Added psycopg2-binary==2.9.12 to requirements.txt.

CREATE EXTENSION needs superuser

Migration's CREATE EXTENSION pg_trgm/unaccent calls require superuser permission. Fix: pre-created the extensions as the postgres user before running migrations.

Incomplete initial migration

0001_initial.py only creates 3 of 9 tables. The missing 6 are auto-created at runtime by FastAPI's lifespan create_tables() call.

ProtectHome=true + asyncpg crash

systemd's ProtectHome=true blocks /home access. asyncpg unconditionally checks $HOME/.postgresql/postgresql.key and crashes with PermissionError. Fix: change to ProtectHome=tmpfs.

SQLAlchemy enum stores NAME not VALUE

Column(Enum(UserRole)) stores 'DOCTOR'/'ADMIN'/'PATIENT' in DB, not 'doctor'/'admin'/'patient'. SQL queries must use uppercase. Long-term fix: Column(Enum(UserRole, values_callable=lambda x: [e.value for e in x])).

Appointment model has no date/time fields

Seed script assumed appointment_date/start_time/end_time. They live on the linked time_slot row, not on Appointment itself.

Review.patient_id FKs to users.id, not patient_profiles.id

Seed script passed the wrong ID. Caused FK constraint violations.

Slot unique constraint on (doctor, date, start_time, type)

Hardcoded time(10, 0) caused collisions when seeding multiple past appointments. Fix: randomized the past slot times.

4.2 Frontend bugs (dr-finder-python-frontend)

Missing src/types.ts entirely

Repo had 9 files importing from '../types' or './types', but no types module existed anywhere. Fix: created src/types.ts with all 18 expected exports (User, DoctorSummary, AvailabilityRequest, etc.).

Capital-cased config files

Tailwind.config.js and Postcss.config.js with capital first letter. Windows is case-insensitive; Linux is not. Vite couldn't find them. Fix: renamed to tailwind.config.js and postcss.config.js.

Build script ran tsc which had project-reference errors

tsconfig.node.json was missing composite: true. Fix: changed package.json's build script from tsc && vite build to just vite build (Vite handles transpilation; type-checking is a separate concern).

SearchPage used keepPreviousData (React Query v4)

React Query v5 removed this option. Resulted in silent fallback to default behavior — not catastrophic but worth removing to silence console warnings.

SearchPage initialized page state to 0

Backend's page: int = Query(1, ge=1) rejects 0 with HTTP 422. Frontend saw empty result and showed 'No doctors found'. Fix: changed all setPage(0) to setPage(1).

HomePage/SpecializationsPage used wrong response shape

API returns array directly, but pages did r.data.data ?? []. Fix: changed to r.data.data ?? r.data ?? [] (matches both shapes).

Backend cached duplicate-name results in Redis

After we fixed the 'Dr. Dr.' duplicate prefix in the database, search results still showed the old names. Fix: redis-cli -a $PASSWORD -n 2 FLUSHDB.

Seed names had 'Dr.' prefix; frontend added another

Result: 'Dr. Dr. Vikram Singh'. Fix: UPDATE users SET full_name = REGEXP_REPLACE(...).

5. The doctor-api systemd Unit

This is the hardened systemd unit that runs the FastAPI backend. The hardening flags would have severely limited the cryptominer's capabilities had it managed to compromise this service:

```
[Unit]
Description=DoctorFinder FastAPI backend
After=network.target postgresql.service redis-server.service
Wants=postgresql.service redis-server.service

[Service]
Type=simple
User=doctor-api
Group=doctor-api
WorkingDirectory=/var/www/doctor/backend
EnvironmentFile=/var/www/doctor/backend/.env
ExecStart=/var/www/doctor/backend/.venv/bin/uvicorn app.main:app \
    --host 127.0.0.1 --port 8002 --workers 1 \
    --proxy-headers --forwarded-allow-ips 127.0.0.1 \
    --log-level info --access-log

Restart=on-failure
RestartSec=5
TimeoutStopSec=20

# === Hardening ===
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=tmpfs        # NOT true — asyncpg needs to stat $HOME
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectKernelLogs=true
ProtectControlGroups=true
ProtectClock=true
ProtectHostname=true
RestrictNamespaces=true
RestrictRealtime=true
RestrictSUIDSGID=true
LockPersonality=true
MemoryDenyWriteExecute=true
RemoveIPC=true
PrivateDevices=true

ReadWritePaths=/var/www/doctor/logs /var/www/doctor/media

SystemCallArchitectures=native
SystemCallFilter=@system-service
SystemCallErrorNumber=EPERM

CapabilityBoundingSet=
AmbientCapabilities=

LimitNOFILE=4096
MemoryMax=300M
TasksMax=50

[Install]
WantedBy=multi-user.target
```

6. nginx Configuration

doctor.vexenlabs.com serves both the static React SPA and proxies /api/* to the FastAPI backend. Note the nginx 1.24 syntax for HTTP/2 — the newer standalone http2 on; directive does NOT work.

```
# HTTP → HTTPS redirect
server {
    listen 80;
    listen [::]:80;
    server_name doctor.vexenlabs.com;
    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://$host$request_uri; }
}

# Main HTTPS server
server {
    listen 443 ssl http2;           # nginx 1.24 syntax
    listen [::]:443 ssl http2;
    server_name doctor.vexenlabs.com;

    ssl_certificate     /etc/letsencrypt/live/doctor.vexenlabs.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/doctor.vexenlabs.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    root /var/www/doctor/frontend/dist;
    index index.html;
    client_max_body_size 10M;

    # Login — strict rate limit
    location = /api/v1/auth/login {
        limit_req zone=login_zone burst=10 nodelay;
        proxy_pass http://127.0.0.1:8002;
        proxy_http_version 1.1;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Other API traffic
    location /api/ {
        limit_req zone=api_zone burst=200 nodelay;
        proxy_pass http://127.0.0.1:8002;
        proxy_http_version 1.1;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    # FastAPI docs
    location ~ ^/(docs|redoc|openapi.json) {
        proxy_pass http://127.0.0.1:8002;
        proxy_set_header Host $host;
    }

    # Hashed static assets — long cache
    location /assets/ {
        access_log off;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
```

```
    # SPA fallback
    location / {
        try_files $uri $uri/ /index.html;
    }

    include /etc/nginx/snippets/security-headers.conf;
}
```

7. Demo State & Credentials

The DoctorFinder demo at https://doctor.vexenlabs.com is seeded with:

Resource	Count
Specializations	6 (Cardiology, Dermatology, General Physician, Pediatrics, Orthopedics, Neurology)
Doctors	8 across 3 cities (Delhi, Mumbai, Bangalore)
Patients	5
Time slots	~2,400 over 30 days
Completed appointments	~40 with reviews
Ratings range	4.10 – 4.70

Demo login credentials (all use password Demo@1234):

Role	Email
`Admin`	`admin@vexenlabs.com`
`Doctor`	`priya.mehta@vexen.demo`
`Patient`	`aakash.verma@vexen.demo`

API documentation (Swagger UI): https://doctor.vexenlabs.com/docs

8. Lessons Learned

Never run app services as a login user

The Node.js service ran as **aslam**, which had a real shell and home directory. When the RCE landed, the attacker inherited shell-level access to that account. Always create a dedicated --system --no-create-home service user with /usr/sbin/nologin.

Outbound deny-by-default is the most underrated firewall rule

Most ufw guides only configure inbound. But by the time malware is on a host, the only way to neutralize it is preventing C2 callback. Outbound deny + explicit allowlist for 53/80/443/123 covers ~99% of legitimate use cases.

Repos can ship broken

The DoctorFinder repo had a missing types module, capital-cased config files (works on Windows, broken on Linux), an incomplete migration, and no alembic config. On a fresh deploy, expect to discover bugs the original devs didn't see because their environment hid them.

Snapshot before every meaningful state change

compromised-evidence-2026-05-14 preserved forensic data. hardened-baseline-2026-05-14 meant the OS hardening was reversibly applied. doctorfinder-live-2026-05-15 means tomorrow's accidental breakage is a 5-minute rollback. The cost is negligible (~$0.01/GB/day on DigitalOcean).

Don't trust 'it builds locally'

Windows/macOS dev environments paper over case-sensitivity, line endings, file mode bits, and PATH differences. The first Linux deploy of a Windows-developed app almost always exposes 5-10 of these. Plan for it.

Push fixes back to the repo immediately

Every fix that lives only on the production server is technical debt waiting to bite the next deploy. Push back the same day, even if it's messy commits.

9. If This Happens Again — Recovery Checklist

Use this checklist if a future production droplet is compromised. Times below are realistic for someone familiar with the process.

0–15 min: Triage

• Take a snapshot named compromised-evidence-YYYY-MM-DD BEFORE doing anything else.
• Identify the running malicious processes (top, ps aux, ss -tnp).
• Pull /tmp/*.txt or any obvious evidence files via scp.
• Do NOT try to clean. The droplet is now write-off.

15–30 min: Decisions

• Notify any stakeholders.
• Identify all credentials the droplet had: DB passwords, API keys, SSH keys.
• List every place those credentials are reused. Rotate them all.

30 min – 2 hr: Rebuild

• Create new droplet, Ubuntu 24.04 LTS, SSH key only.
• Apply Section 2 hardening (SSH, UFW with outbound deny, fail2ban, unattended-upgrades, sysctl).
• Take snapshot hardened-baseline-YYYY-MM-DD.

2–6 hr: Reinstall apps

• Recreate service users with non-login shells.
• Recreate databases with new passwords (NOT the rotated ones — fresh ones).
• Restore application code from GitHub (NOT from old droplet — could be tampered).
• Restore data from clean backups only. If you don't have clean backups, this is a teachable moment for next time.

6+ hr: Verify & monitor

• Smoke-test critical flows.
• Watch logs for 24h: journalctl -f, tail -f /var/log/auth.log, sudo ufw status verbose.
• Take snapshot recovered-live-YYYY-MM-DD.

— End of runbook —